MessageFoundry™

v0.3.2 · July 2026

MESSAGEFOUNDRY DOCUMENTATION

Windows service

MessageFoundry runs as a long-lived background service via NSSM (the "Non-Sucking Service Manager"). NSSM wraps the existing messagefoundry serve command: it starts the engine on boot, restarts it on crash, and captures its output to rotating log files.

Generated from `docs/SERVICE.md@abedf865`

MessageFoundry runs as a long-lived background service via NSSM (the "Non-Sucking Service Manager"). NSSM wraps the existing `messagefoundry serve` command: it starts the engine on boot, restarts it on crash, and captures its output to rotating log files. Stopping the service sends Ctrl+C so the engine drains its connections cleanly (the ASGI lifespan calls `engine.stop()`).

This is the localhost, single-machine setup: the service binds `127.0.0.1` and authentication is already required (it is on by default). Networked deployment is **built but deliberate** — bind off-loopback with `[security].local_access_only = false` + `[security].listen_address`, and an off-loopback API bind is *refused at startup* unless TLS is configured (`[api].tls_cert_file`, or a trusted upstream terminator). See DEPLOYMENT.md for the exposure checklist and REMOTE-CONSOLE.md for operating this service from another PC.

Before going live, hand your endpoint-security and firewall admins the Antivirus Exclusions & Firewall Permissions guide: it spells out the narrow set of AV exclusions (the SQLite store + `-wal` / `-shm` sidecars, logs, key/cert files, the venv interpreter) and the per-connection firewall openings the service needs.

Prerequisites

1. **Python venv with the package installed.** From the repo root: `powershell python -m venv .venv .venv\Scripts\python.exe -m pip install -e .` This puts `messagefoundry.exe` in `.venv\Scripts\` — the service points at it. For a **reproducible, pinned** deployment, install the locked, hash-verified dependency set first, then the package itself: `powershell .venv\Scripts\python.exe -m pip install --require-hashes -r requirements.lock .venv\Scripts\python.exe -m pip install -e . --no-deps` `requirements.lock` is the SHA-256-pinned export checked in sync and audited in CI (DEP-1).
2. **NSSM** — provisioned automatically. If `nssm.exe` isn't on `PATH` (or passed via `-NssmPath`), `install-service.ps1` downloads the pinned, SHA-256-verified release into `<DataDir>\bin\nssm.exe` and uses it. No manual install needed. (You can still pre-install it — `choco install nssm` or a download from https://nssm.cc — and it'll be used if found.)
3. **An elevated PowerShell** (Run as Administrator) — required to register a service. `messagefoundry service install --env <name>` elevates for you (a UAC prompt) and runs the install script below in a visible window so you can read its output.

Install

```
# from repo root, elevated PowerShell
.\scripts\service\install-service.ps1 -Environment prod
```

`-Environment` is **required** (ADR 0017): it selects which `environments/<name>.toml` value file the engine resolves and the instance's PHI posture. `serve` refuses to start without it (no silent default), so the install script refuses too — pass `dev`, `staging`, `prod`, or a custom name. **A custom name must also declare its posture** — `[security].handles_real_patient_data` and `[security].production_instance` in the service config (`messagefoundry.toml`) — because a free-form environment name carries no PHI posture to derive one from (only `dev` / `staging` / `prod` do). Without both, `serve` exits 2 with "*environment* '\<*name*>'

has no built-in security posture", so the service registers fine and then dies on every start. (The pre-ADR-0118 spellings `[ai].data_class` / `[ai].production` are **rejected at config load** — see ADR 0118.) `messagefoundry service install` requires the same name via `--env` and passes it straight through.

Defaults:

Setting	Default
Service name	`MessageFoundry`
Engine exe	`<repo>\.venv\Scripts\messagefoundry.exe`
Config dir	`<repo>\samples\config`
Active environment	*(required — `-Environment`)*
Data dir	`C:\ProgramData\MessageFoundry`
Message store	`<DataDir>\messagefoundry.db`
Logs	`<DataDir>\logs\service.out.log`, `service.err.log`
Bind	`127.0.0.1:8765`
Log level	`INFO`

Override any of them, e.g.:

```
.\scripts\service\install-service.ps1 -Environment prod -Port 9000 -LogLevel DEBUG `
    -Config D:\hl7\config -DataDir D:\MessageFoundry
```

The install script is idempotent — re-running it reconfigures the existing service.

> **Migration note (ADR 0050, `--project-root`).** `serve` / `supervise --project-root R` now anchors the **whole** config bundle under `R` — the `--config` graph, `environments/<env>.toml`, `messagefoundry.toml`, **and** a relative `[store].path` / `--db` (and each shard's `<stem>_<shard>.db`). Previously only `environments/` was anchored; a relative DB resolved against the process CWD. If you already run with `--project-root` **and** a relative DB path, the DB will now be found/created under `R` — pass an **absolute** `[store].path` / `--db` to keep it in place (absolute paths bypass the root), or accept the new location. The startup CWD-mismatch WARNING names the resolved paths so any move is visible. Deployments without `--project-root`, or with an absolute DB path, are unchanged.

Update to a new build (restart vs reinstall)

The service runs `<repo>\.venv\Scripts\messagefoundry.exe`. With the documented **editable** install (`pip install -e .`), that exe imports straight from the repo source — so a running service keeps the code it loaded **at process start**. To pick up new code (a pull, a branch switch, a merge), just **restart** it (elevated):

```
& C:\ProgramData\MessageFoundry\bin\nssm.exe restart MessageFoundry
curl http://127.0.0.1:8765/health
```

Because the install is editable, a restart runs **whatever branch is checked out** in the repo.

Reinstall instead when paths or flags change (port, config dir, data dir) or the service definition drifted — the install script is idempotent (it stops and reconfigures in place):

```
.\scripts\service\install-service.ps1 -Environment prod   # elevated; re-points the exe + AppParameters
& C:\ProgramData\MessageFoundry\bin\nssm.exe start MessageFoundry
```

If the package was installed **non-editable** (a plain `pip install .`), the venv holds a snapshot of the old code — run `.venv\Scripts\python.exe -m pip install -e .` first, then restart.

> ⚠ **Upgrading from ≤ 0.2.5 → ≥ 0.2.6: tighten the config-dir ACLs first.** The config-directory permission guard (SEC-003 / ADR 0036) did **not** exist in 0.2.5. From 0.2.6 on, `serve` refuses to start against a `--config` directory **writable by a broad principal** (e.g. `Authenticated Users` / `S-1-5-11`) — "*refusing to load config from writable-by-others path …*". A config dir that inherits that write (common under `C:\srv\…`) therefore **fails its first start after the upgrade**. Lock it down **before** restarting onto the new build — use the surgical recipe under Lock down the config directory (CONFIG-2) (`icacls /inheritance:d /T` + `/remove:g *S-1-5-11 /T` + grant SYSTEM/Admins), the lighter-touch alternative to a full `/inheritance:r` reset for a shared tree.

Start / stop / status

```
nssm start  MessageFoundry
nssm status MessageFoundry
nssm stop   MessageFoundry      # Ctrl+C -> graceful connection shutdown (up to 15s)
nssm restart MessageFoundry
```

If `nssm` isn't on `PATH`, it's the auto-downloaded copy at `<DataDir>\bin\nssm.exe` (e.g. `C:\ProgramData\MessageFoundry\bin\nssm.exe`). You can also use the built-in `sc.exe` / Services.msc once installed.

For a one-click desktop alternative to these commands — engine status at a glance plus start/stop/restart, the console, and the log from the notification area — run the **Windows tray service-manager** (`messagefoundry-tray` — shipped in the wheel, on every install).

Security hardening (recommended)

Run as a least-privilege account (DEPLOY-1)

By default the service now installs under a **least-privilege per-service virtual account** — `NT SERVICE\<ServiceName>` (e.g. `NT SERVICE\MessageFoundry`), which needs **no password** (#224). A bare install

therefore lands on the least-privilege posture with nothing extra to configure:

```
.\scripts\service\install-service.ps1 -Environment prod   # runs as NT SERVICE\MessageFoundry
```

The engine needs only **read** on the config directory and **read/write** on the data directory, so the virtual account grants exactly what is required. The installer **auto-grants the account what it needs**: read+execute on the config directory and read/write on the data directory — and, because a per-service virtual-account SID does not resolve until the service exists, those ACLs are applied **after** the service is registered and its run-as account is set (the manual `icacls` lines below are only needed if you point the engine at directories outside those). Running as **LocalSystem** — the most privileged local account — widens the blast radius of any compromise (a config module is executed in-process — see *Lock down the config directory* below), so it is now an explicit opt-out:

```
.\scripts\service\install-service.ps1 -Environment prod -AllowLocalSystem   # opt out to
LocalSystem
```

To run under a **different** account (a domain gMSA, a dedicated local user, or another virtual account), pass `-ServiceAccount` — it always wins over the default:

```
.\scripts\service\install-service.ps1 -Environment prod -ServiceAccount "NT
SERVICE\MessageFoundry"
```

```
# only if config/data live outside the script-managed paths:
icacls "D:\hl7\config"                /grant "NT SERVICE\MessageFoundry:(OI)(CI)RX"
icacls "C:\ProgramData\MessageFoundry" /grant "NT SERVICE\MessageFoundry:(OI)(CI)M"
```

A domain **gMSA** or a dedicated local user works the same way (pass `-ServiceAccountPassword` for a password-based account — it's taken as a `SecureString`). The store file itself is further restricted to its owner at runtime; account choice governs who that owner is.

gMSA preflight + "Log on as a service" (#99). When `-ServiceAccount` names a **gMSA** (a name ending in `$`, e.g. `CORP\mefor-svc$`), the installer runs `Test-ADServiceAccount` first (verifying the account is installed + usable on this host — run `Install-ADServiceAccount <name>` if not) and then grants the account `SeServiceLogonRight` ("Log on as a service") via `secedit` before registering — without that right the service fails to start with **error 1069**. Both steps are **best-effort and degrade gracefully**: on a non-domain / RSAT-less dev box the preflight skips with a message and the install proceeds; neither ever aborts. Pass `-SkipGmsaPreflight` when the account is provisioned by a separate runbook. The end-to-end SQL-Server-integrated-auth gMSA walkthrough (grant the gMSA a SQL login, `[store].auth = "integrated"`) is in DEPLOY-SERVER-DB.md §1.1.

Default flipped to least-privilege; `-AllowLocalSystem` is the LocalSystem opt-out (#224, built on #99). Omitting both `-ServiceAccount` and `-AllowLocalSystem` now installs under the virtual account `NT SERVICE\<ServiceName>` — **not** LocalSystem. To run as LocalSystem you must pass `-AllowLocalSystem` explicitly (it prints a warning that LocalSystem is the acknowledged, non-default choice). An existing

unattended install that already passed `-ServiceAccount` is unaffected; one that relied on the old LocalSystem default now needs `-AllowLocalSystem` to keep running as LocalSystem — the recommended migration is to accept the new virtual-account default instead. This flip is exercised end-to-end by the `windows-service-smoke` CI leg (a bare `-LockConfigDir` install, which now runs under the virtual account, must start and serve `/health` + MLLP on both Windows Server SKUs).

Protect the store encryption key at rest (WP-11d)

PHI columns are AES-256-GCM-encrypted at rest when a key is configured (see PHI.md §3). The key is a base64 32-byte secret. Two ways to supply it:

- **Environment (cross-platform default).** Set `MEFOR_STORE_ENCRYPTION_KEY` in the service's environment (`nssm set MessageFoundry AppEnvironmentExtra MEFOR_STORE_ENCRYPTION_KEY=...`). Simple, but the plaintext key sits in the service environment block, readable by any local administrator.
- **DPAPI-protected key file (Windows).** Keep the key in a file that Windows DPAPI binds to *this machine*, so a copied file is useless elsewhere and no plaintext key is in the environment:

`powershell # mint + protect a fresh key (machine scope, so the service account can read it at startup). # SYSTEM is granted read automatically (covers a LocalSystem service); for a virtual / gMSA service # account add --grant-account '<that account>' so the service — not just you — can read the key: messagefoundry protect-key --generate --out "C:\ProgramData\MessageFoundry\store.key.dpapi" # (virtual account example: ... --grant-account "NT SERVICE\MessageFoundry") # -> prints the base64 key ONCE to stderr; back it up offline (the file is machine-bound and # unrecoverable if the host is lost), then point the engine at it:` `toml [store] encryption_key_file = "C:/ProgramData/MessageFoundry/store.key.dpapi"` Then **unset** `MEFOR_STORE_ENCRYPTION_KEY` (the env key takes precedence when both are set). The service account `CryptUnprotectData`s the file at startup; a missing/foreign/unreadable file makes `serve` fail closed rather than store PHI unencrypted. `protect-key` locks the file to the minting admin **plus** the service principal it grants read — SYSTEM by default, or `--grant-account` for a virtual / gMSA account. It sets an explicit DACL with inheritance **disabled**, so the file does **not** inherit the data-dir ACL — grant the right service account at mint time (above) rather than relying on the directory. To rotate, `protect-key` a new key to the file and run `messagefoundry rotate-key` with the prior key in `MEFOR_STORE_ENCRYPTION_KEYS_RETIRED` (see PHI.md §3).

> **External vault / managed identity.** DPAPI is the built-in on-box option. To source the key (or SQL/AD credentials) from an external secrets manager — Windows Credential Manager, HashiCorp Vault, Azure Key Vault via a **managed identity**, or an AD **gMSA** for SQL/LDAP — fetch the secret in your service-start wrapper and export it as the corresponding `MEFOR_*` variable, or place the DPAPI key file via your provisioning tool. The engine reads only env/`encryption_key_file`; it does not call a vault directly (a thin broker is future work).

Lock down the config directory (CONFIG-2)

`messagefoundry serve --config <dir>` and `POST /config/reload` **execute the Python** in the config directory in-process, with the service account's privileges. The directory is therefore a trust boundary: anyone who can write a `.py` file there can run code as the service.

- Restrict the config directory's ACL so only administrators / the service account can write it: `powershell icacls "D:\hl7\config" /inheritance:r /grant "Administrators:(OI)(CI)F" "NT SERVICE\MessageFoundry:(OI)(CI)R"` The supported one-step way to do this at install time is `install-service.ps1 -LockConfigDir`: it strips inherited ACEs and locks the dir to SYSTEM + Administrators (full) and the run-as account (read+execute) — no companion flag needed, since the run-as account defaults to the virtual account. It is **opt-in** because the config dir often lives inside a developer's repo where stripping inheritance is surprising — for production, point `-Config` at a dedicated admin-owned directory and pass `-LockConfigDir`.
- **Fix an existing tree that inherits a broad write grant** without a full ACL reset. A config dir placed under a shared root (e.g. `C:\srv\…`) often inherits `Authenticated Users` (`S-1-5-11`) write, which trips the guard below. The lighter-touch alternative to `/inheritance:r` is to break inheritance, surgically drop just the broad principal, and grant the run-as user read+execute: `powershell icacls "C:\srv\mefor\config" /inheritance:d /T icacls "C:\srv\mefor\config" /remove:g *S-1-5-11 /T icacls "C:\srv\mefor\config" /grant "<run-as-user>:(OI)(CI)RX" /T`
- The loader **actively enforces** this at load time (and on `/config/reload`), not just as a documented recommendation (ADR 0036, SEC-003):
- On **Windows** the loader now parses the directory's and each `*.py`'s NTFS owner + DACL and **refuses** to load when a broad/low-privilege principal (Everyone, Authenticated Users, `BUILTIN\Users`, INTERACTIVE, …) or any non-owner/non-admin principal holds a write-class right (write/append/delete/`WRITE_DAC`/`WRITE_OWNER`/generic-write). A `NULL` DACL (everyone allowed) is likewise refused. If the DACL **cannot be read** (a Win32 API error), the guard **fails open with a loud WARNING** rather than bricking a previously-working service — a WARNING about an *unevaluable* guard means "fix/lock the config-dir ACL", not "ignore it".
- On **POSIX** hosts the loader **refuses** to load from a group/world-writable or foreign-owned directory or module file.
- **Dev/test escape (never set in production).** Because a default Windows checkout grants `BUILTIN\Users` write, set `MEFOR_ALLOW_INSECURE_CONFIG_SOURCE=1` to downgrade the refusal to a loud WARNING when running from an intentionally user-writable dev/CI tree. A production service leaves it unset and locks the config dir (above), so the guard stays fail-closed; the env var is the explicit, audited opt-out (mirrors `MEFOR_ALLOW_INSECURE_TLS`).
- `/config/reload` only loads from the startup `--config` directory and any directories listed in `[api].config_reload_roots` (see CONFIGURATION.md); an arbitrary path is rejected. Keep those roots admin-owned too.

Suppress Windows crash dumps of the engine (ADR 0152 Phase 0)

A Windows Error Reporting crash dump of the engine is a **full PHI disclosure written to disk**: in-flight HL7 bodies, the plaintext the store cipher just produced, and the unwrapped DEK all live in process memory, and a dump copies that memory into a file that is **not** the store — not encrypted, not covered by the store's ACLs, and never touched by the retention sweep.

The engine applies the **process-local** half of the fix on every `serve`, unconditionally and with no configuration: `SetErrorMode` (ORed into the inherited mode, so it can never clear a protection you set upstream) plus `WerSetFlags(NOHEAP | NO_UI | DISABLE_SNAPSHOT_CRASH | DISABLE_SNAPSHOT_HANG)`. It persists nothing and needs no privilege — a `serve` invocation never silently rewrites machine state.

The other half is **machine policy and no process can reach it**: `HKLM\…\Windows Error Reporting\LocalDumps` is evaluated independently of the WER exclusion list and of every flag a process can set, so a LocalDumps-configured host still dumps a process that suppressed everything available to it. Close it at install time:

```
.\install-service.ps1 -Environment prod -SuppressCrashDumps
```

It is **opt-in** because it writes `HKLM` keys **by image name**, which affects every process of that name on the host — an explicit operator decision, not a side effect of installing. It registers both `messagefoundry.exe` and the venv `python.exe`: a pip console-script launcher starts the interpreter as a **child process**, so the process holding the PHI heap (and the one WER would dump) is `python.exe`, and naming only the launcher would produce a policy that looks applied and protects nothing.

`LocalDumps` is only ever narrowed, never switched on. WER local dump collection is **opt-in**: if the `LocalDumps` key does not exist, no local dumps are collected for anything on the host. Creating `LocalDumps\<image>` where the parent key was absent would therefore *configure* per-image dump collection where there was none — a `-SuppressCrashDumps` switch that plausibly **enables** PHI dumps is worse than no switch at all. So the installer writes the per-image override **only when a `LocalDumps` configuration already exists**, and reports which of the two it did. Where it does write it, the override is `DumpType=0` + `CustomDumpFlags=0` (`MiniDumpNormal` — **no heap**, which is the PHI-relevant part) so a host configured for full dumps stops writing the engine's heap. Note that Microsoft documents `DumpCount` as *the maximum number of dump files in the folder*, **not** as a disable switch, so treat the override as a **reduction, not an elimination**: a `MiniDumpNormal` still carries thread stacks. To eliminate local dumps entirely, remove the host's `LocalDumps` configuration.

Residuals, stated plainly. A postmortem debugger registered under `AeDebug` attaches ahead of WER entirely and is outside both halves. The keys **persist after `uninstall-service.ps1`** (deliberately — silently re-enabling PHI dumps on uninstall would be the more surprising default); remove them by hand under that registry path to restore the host's original behaviour. Neither half has been verified by forcing an actual crash with PHI resident and inspecting the dump directory — the Win32 calls were confirmed to succeed, the registry writes were not executed. And none of this bears on ASVS 11.7.1: memory **hygiene** is not memory **encryption**.

Verify it's running

```
curl http://127.0.0.1:8765/health        # -> {"status":"ok", ...}
```

Send a test message and confirm it flows through:

```
.venv\Scripts\python.exe samples\send_mllp.py samples\messages\adt_a01.hl7
```

Then check the log:

```
Get-Content C:\ProgramData\MessageFoundry\logs\service.out.log -Tail 20 -Wait
```

You should see uvicorn's startup banner and `wiring started: N inbound, N outbound connection(s)`. On stop you should see `wiring stopped` and `engine stopping` — confirmation of a clean shutdown. (A live config swap logs `wiring reloaded: ...`.)

Logs

The engine logs to stdout/stderr with a stdlib `logging` setup (one timestamped UTC stream — see `messagefoundry/logging_setup.py`), with a CR/LF log-injection filter and a `safe_exc()` PHI-redaction chokepoint on the exception path (WP-6c — see PHI.md §7). NSSM captures those streams to the files above and rotates them at ~10 MB. Structured (JSON) logging + off-box (syslog/SIEM) forwarding are planned (bundled with off-box exposure); until then **avoid raising the level to `DEBUG` in production**, since verbose output may include message content.

Restrict the log directory's ACL so the captured stdout/stderr (operational data, not message bodies) is readable only by administrators and the service account — NSSM's files would otherwise inherit a broadly-readable `ProgramData` ACL (ASVS 16.4.2):

```
icacls "C:\ProgramData\MessageFoundry\logs" /inheritance:r `
  /grant "Administrators:(OI)(CI)F" "NT SERVICE\MessageFoundry:(OI)(CI)M"
```

Admin console (in a browser)

This service is **headless**. Operators watch and run it from the **browser web console** served same-origin at `/ui` (not part of the service runtime — a separate, version-matched wheel the engine mounts in-process). The wheel is **not published to an index yet**, so install it by path:

```
pip install -e packaging/messagefoundry-webconsole   # into the engine venv
```

No switch is needed: the console is **on by default** for a loopback bind (ADR 0143), and `[security].serve_web_console = false` turns it off. (`[api].serve_ui` was the old spelling and is now **refused at config load** — ADR 0118 moved the console/bind/origin switches into `[security]`.)

Browse to this service's `/ui` (`http://127.0.0.1:8765/ui`) and sign in. See INSTALL-GUIDE.md → "Launching the admin console". (The former PySide6 desktop console was retired — BACKLOG #103.)

High-delivery-rate TCP tuning (engine host)

Outbound MLLP ships **connect-per-message as the default this release** (`persistent=false` — today's proven posture; ADR 0067 §8), so on the default posture the engine opens **one TCP connection per delivered message**. At sustained delivery rates in the hundreds of messages per second the engine host is the active closer of every one of those connections, so closed sockets accumulate in `TIME_WAIT` and can exhaust the **default Windows ephemeral port range (16,384 ports)** — deliveries then dead-letter with `MLLP connect ... failed` even though the partner is healthy. This was measured on the 2026-07-02 load campaign: `TIME_WAIT` peaked above the default range and ~50% of deliveries dead-lettered; with the tuning below the same load ran without exhaustion. This applies to the Pilot/Standard tiers only at high per-lane rates — most on-prem feeds never approach it.

The durable fix is `persistent=true` (ADR 0067), a documented **opt-in** this release: it reuses one connection per destination, removing the per-message handshake and the `TIME_WAIT` accumulation entirely. Operators running sustained high-rate delivery can opt in per outbound today (`persistent = true` on the `MLLP()` destination); the default flips to `persistent=true` in a subsequent release once the ADR 0067 §8 trigger is met. Until you opt in — or the default flips — apply the host tuning below on high-rate lanes.

If you run sustained delivery in the hundreds of msg/s on the default (`persistent=false`) posture (or see connect-failure dead-letters while partners are reachable), either opt into `persistent=true` on that outbound, or widen the range and shorten the wait (administrator PowerShell, engine host only):

```
# Widen the ephemeral range (here: 22000-65535 ~= 43,500 ports)
netsh int ipv4 set dynamicport tcp start=22000 num=43535
# Halve how long a closed socket lingers (default 120s on older builds, 60s on newer)
Set-ItemProperty -Path 'HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters' `
  -Name TcpTimedWaitDelay -Value 30 -Type DWord   # reboot to apply
```

Check current state: `netsh int ipv4 show dynamicport tcp` and `(Get-NetTCPConnection -State TimeWait).Count`. Revert by restoring the defaults (`start=49152 num=16384`, delete the `TcpTimedWaitDelay` value). Both changes are host-wide — coordinate with whatever else the box runs.

Uninstall

```
.\scripts\service\uninstall-service.ps1
```

This stops and removes the service. The log files and message store under `DataDir` are left in place.

Troubleshooting

- **Service won't start / exits immediately.** Read `service.err.log`. The most common cause is a bad path baked into the service (relative paths resolve to the *system* directory for a service account); re-run the install script, which resolves all paths to absolute.
- **Port already in use (e.g. 2575).** The sample config's inbound connection binds MLLP port `2575`. If a stray `messagefoundry serve` (or a second copy of the service) is already running, the listener fails to bind. Make sure only one instance runs: `Get-Process messagefoundry,python | Format-Table Id,ProcessName,Path`.
- **`/health` doesn't respond.** Confirm the service is `SERVICE_RUNNING` (`nssm status MessageFoundry`) and that nothing else owns port `8765`.
- **Permissions on the data dir.** The service runs by default as the least-privilege virtual account `NT SERVICE\<ServiceName>`, to which the installer grants read/write on `C:\ProgramData\MessageFoundry` (after registration, once the per-service SID resolves). If you point `-DataDir` somewhere outside the script-managed path, grant that account read/write there too (the installer only ACLs the default data dir), or startup fails — pick a writable location and grant it, or opt out to LocalSystem with `-AllowLocalSystem` (see *Security hardening* above).